

Reviewer Pack — Cut-Polynomial Kurtosis Excess Bounds GW SDP Integrality Gap

Entry 485f006f57f6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 10:10:53 UTC

Cut-Polynomial Kurtosis Excess Bounds GW SDP Integrality Gap

Entry ID: 485f006f57f6 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 10:10:53 UTC

1. Conjecture

Field A (mathematical branch): Fourier-analytic hypercontractivity on the Boolean cube (Bonami-Beckner (4,2)-norm comparison and excess-kurtosis of low-degree polynomials in the KKL/Friedgut/Mossel tradition) **Field B** (complexity object): Goemans-Williamson / SOS-degree-2 (Delorme-Poljak) SDP integrality gap for unweighted Max-Cut on 3-regular graphs

Statement:

For every connected 3-regular graph G on n vertices with adjacency matrix A , define the cut polynomial $g_G(x) := x^T A x$ for $x \in \{\pm 1\}^n$ and the Bonami-Beckner kurtosis excess $\kappa(G) := E_x[g_G(x)^4] / E_x[g_G(x)^2]^2 - 3$, where E_x is uniform over $\{\pm 1\}^n$. Then the Goemans-Williamson / SOS-2 integrality gap obeys $(DP(G) - \text{MaxCut}(G)) / |E(G)| \leq 0.5 \cdot \max(\kappa(G), 1/n)^{1/4}$, where $DP(G) = |E|/2 + n \cdot |\lambda_{\min}(A)|/4$ is the Delorme-Poljak eigenvalue SDP bound. A single 3-regular G with $n \leq 20$ violating this inequality falsifies the conjecture.

Rationale (proposer's reasoning):

Goemans-Williamson rounding succeeds exactly when the SDP-induced quadratic form g_G behaves Gaussianly under random ± 1 inputs; the Bonami-Beckner (4,2)-hypercontractive excess kurtosis κ is the canonical degree-2 deviation-from-Gaussianity invariant and is therefore a natural Fourier-analytic certificate for SDP tightness. Since g_G is degree-2, its full Fourier mass sits at level ≤ 2 , so κ measures purely the 4th-cumulant cross-edge structure that GW rounding fails to capture. If true, this gives a polynomial-time computable moment-matrix-style witness $P(M) = \kappa$ that any 0.879-approximator must respect, transferable under polynomial reductions that preserve quadratic structure.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e217b8fe14bdf1cc

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 4 sizes $n \in \{10, 14, 18, 20\} \times 30$ seeds = 120 random 3-regular graphs, every instance must satisfy $(DP(G) - \text{MaxCut}(G))/|E(G)| \leq 0.5 \cdot \max(\kappa(G), 1/n)^{1/4}$, AND the Spearman ρ between $(DP - \text{MaxCut})/|E|$ and $\max(\kappa, 1/n)^{1/4}$ across the 120 instances must be ≥ 0.4 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.86	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Max-Cut SDP integrality gap 3-regular hypercontractivity kurtosis - Delorme-Poljak eigenvalue bound Goemans-Williamson cubic graph Bonami-Beckner - low-degree polynomial fourth moment Boolean cube Max-Cut SOS degree-2 gap

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        # Find pivot
        max_row = i
        for k in range(i+1, n):
            if abs(matrix[k][i]) > abs(matrix[max_row][i]):
                max_row = k
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        # Eliminate below the pivot
        factor = 1 / matrix[i][i]
        for j in range(n):
            if j != i:
                matrix[j][i] *= factor

        # Eliminate above the pivot
        for j in range(i+1, n):
            factor = matrix[j][i]
```

```

        for k in range(n):
            matrix[j][k] -= factor * matrix[i][k]
    return matrix

def max_cut(G):
    n = len(G)
    best_cut_value = 0
    for cut in combinations(range(n), n//2):
        cut_value = sum(G[u][v] if (u, v) in G else G[v][u] for u in cut for v in cut if (u, v) not in cut)
        best_cut_value = max(best_cut_value, cut_value)
    return best_cut_value

def bonami_beckner_kurtosis(G):
    n = len(G)
    K = 20000
    sum_g_G_x = 0
    sum_g_G_x_squared = 0
    for _ in range(K):
        x = [random.choice([-1, 1]) for _ in range(n)]
        g_G_x = sum(x[i] * G[i][j] if (i, j) in G else x[j] * G[i][j] for i, j in combinations(range(n), 2))
        sum_g_G_x += g_G_x
        sum_g_G_x_squared += g_G_x ** 2
    E_g_G_x = sum_g_G_x / K
    E_g_G_x_squared = sum_g_G_x_squared / K
    return (E_g_G_x**4 / E_g_G_x_squared**2) - 3

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 10 + (seed % 5) * 4 # n ∈ {10, 14, 18, 20}
    G = {}
    degree_sum = 0
    while degree_sum < 3 * n:
        u, v = random.sample(range(n), 2)
        if (u, v) not in G and (v, u) not in G:
            G[(u, v)] = 1
            degree_sum += 2

    max_cut_value = max_cut(G)
    A = [[G.get((i, j), 0) for j in range(n)] for i in range(n)]
    DP_G = n / 2 + n * min(eigenvalue.real for eigenvalue in gaussian_elimination(A).diagonal()) / n
    kappa_G = bonami_beckner_kurtosis(G)

    lhs = (DP_G - max_cut_value) / len(G)
    rhs = 0.5 * max(kappa_G, 1/n)**(1/4)

    return {
        "metric_name": "(DP-G) - MaxCut",
        "metric_value": lhs,
        "instances_tested": 1,
        "conjecture_holds": lhs <= rhs,
        "counterexample": "" if lhs <= rhs else f"Counterexample for n={n}, seed={seed}"
    }

if __name__ == "__main__":
    import sys

```

```

seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30)) # Default to first 30 primes if

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"First failing seed\" first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dcfa6a46.py", line 96, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dcfa6a46.py", line 74, in run_trial
    max_cut_value = max_cut(G)
                    ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dcfa6a46.py", line 44, in max_cut
    cut_value = sum(G[u][v] if (u, v) in G else G[v][u] for u in cut for v in cut if (u, v) not in G and (v, u) not in G)
                  ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dcfa6a46.py", line 44, in <genexpr>
    cut_value = sum(G[u][v] if (u, v) in G else G[v][u] for u in cut for v in cut if (u, v) not in G and (v, u) not in G)
                  ~~~~~^
KeyError: 0

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `KeyError` in the `max_cut` routine before any instances were evaluated, so neither the per-instance bound nor the Spearman correlation criterion was assessed. | next: Fix the `max_cut` implementation (use `networkx` edge iteration or a correct cut-edge sum over `G.edges`) and rerun the 120-instance protocol.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	228192
2	preregistration	claude_max	opus	0	0	8742
3	novelty	claude_max	opus	0	0	3798
4	novelty	claude_max	opus	0	0	7981
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14526
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12725
7	judge	claude_max	opus	0	0	4603

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 280568 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/485f006f57f6.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/485f006f57f6.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/485f006f57f6.tar.gz` (if generated)